

```
int(fileSize), syscall.PROT_READ, syscall.MAP_SHARED)
    if err != nil {
        panic(err)
    }

    return data
}
```

Parallel processing in GoLearn

Go's built-in concurrency model (goroutines) allows models to train and predict in parallel, improving efficiency.

Train models in parallel: Instead of training models sequentially, use goroutines to train them simultaneously.

```
import (
    "sync"
    "github.com/sjwhitworth/golearn/trees"
    "github.com/sjwhitworth/golearn/knn"
)

func main() {
    var wg sync.WaitGroup

    // Load training data
    trainingData := loadData("train.csv")

    wg.Add(2) // Two goroutines

    // Train Decision Tree in parallel
    go func() {
        defer wg.Done()
        tree := trees.NewID3DecisionTree(0.6)
        tree.Fit(trainingData)
    }()

    // Train KNN classifier in parallel
    go func() {
        defer wg.Done()
        knnClassifier := knn.NewKnnClassifier("euclidean",
            "linear", 3)
        knnClassifier.Fit(trainingData)
    }()

    wg.Wait() // Wait for both tasks to finish
}
```

Parallelising predictions

```
func parallelPredict(models []Model, testData base.
FixedDataGrid) []base.FixedDataGrid {
    var wg sync.WaitGroup
    predictions := make([]base.FixedDataGrid, len(models))
}
```

```
for i, model := range models {
    wg.Add(1)
    go func(i int, model Model) {
        defer wg.Done()
        predictions[i], _ = model.Predict(testData)
    }(i, model)
}

wg.Wait()

return predictions
}
```

Best practices for model evaluation and optimisation

Cross-validation for reliable evaluation: GoLearn provides a `CrossValidate()` function to perform k-fold cross-validation, improving model reliability.

```
import "github.com/sjwhitworth/golearn/evaluation"

// Perform 5-fold cross-validation
cv, err := evaluation.CrossValidateModel(model, data, 5)
if err != nil {
    fmt.Println("Error:", err)
    return
}

fmt.Println("Cross-Validation Accuracy:", cv)
```

Hyperparameter optimisation: Tuning model parameters can significantly improve performance. Try different hyperparameter values for better results.

```
// Example: Trying different K values in KNN
bestK := 1
bestAccuracy := 0.0

for k := 1; k <= 10; k++ {
    knnModel := knn.NewKnnClassifier("euclidean", "linear",
k)
    knnModel.Fit(trainingData)

    // Evaluate accuracy
    accuracy, _ := evaluation.GetAccuracy(knnModel.
Predict(testData), testData)
    if accuracy > bestAccuracy {
        bestAccuracy = accuracy
        bestK = k
    }
}
```